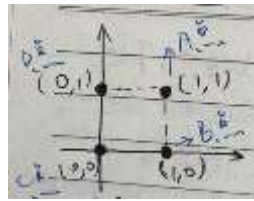


01. 请将圈红圈的三点分别代入进行计算，验证左图中模型和右图中的分类结果是否一致。



验证点 $(1,0)$, $x_1=1, x_2=0$

$$z_1 = x_1 \cdot 1 + x_2 \cdot (-1) - 0.5 = 0.5$$

$$a_1 = \text{syn}(0.5) = 1$$

$$z_2 = x_1 \cdot (-1) + x_2 \cdot 1 - 0.5 = -1.5$$

$$a_2 = \text{syn}(z_2) = 0$$

$$\Rightarrow y = \text{syn}(a_1 x_1 + a_2 x_2 - 0.5) = \text{syn}(0.5) = 1, \text{ 为正类}$$

$\Rightarrow$ 在蓝带内侧 $\checkmark$

验证点 $(0,0)$, $x_1=0, x_2=0$

$$z_1 = x_1 \cdot 1 + x_2 \cdot (-1) - 0.5 = -0.5$$

$$a_1 = \text{syn}(-0.5) = 0$$

$$z_2 = x_1 \cdot (-1) + x_2 \cdot 1 - 0.5 = -0.5$$

$$a_2 = \text{syn}(-0.5) = 0$$

$$\Rightarrow y = \text{syn}(a_1 x_1 + a_2 x_2 - 0.5) = \text{syn}(-0.5) = 0, \text{ 为负类}$$

验证点 $(0,1)$, $x_1=0, x_2=1$ $\Rightarrow$ 在蓝带“外侧”

$$z_1 = x_1 \cdot 1 + x_2 \cdot (-1) - 0.5 = -1.5$$

$$a_1 = \text{syn}(-1.5) = 0$$

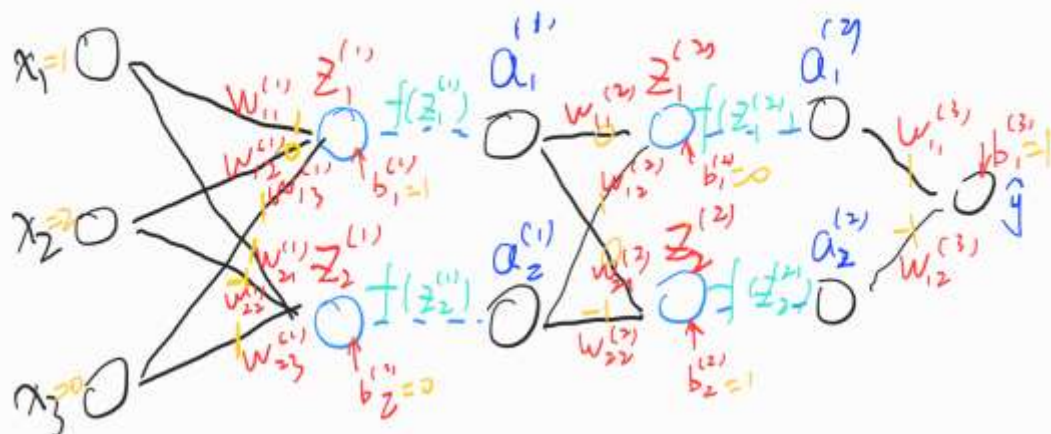
$$z_2 = x_1 \cdot (-1) + x_2 \cdot 1 - 0.5 = 0.5$$

$$a_2 = \text{syn}(0.5) = 1$$

$$\Rightarrow y = \text{syn}(a_1 x_1 + a_2 x_2 - 0.5) = \text{syn}(0.5) = 1, \text{ 为正类, 在“内侧”}$$

02. 题目如图，①标出图中a、w、b；②求所有的Z、a及模型的输出的值。

① 标出 a, w, b



② 求 z, a 及模型输出值。

$$z_1^{(1)} = w_{11}^{(1)} \cdot x_1 + w_{12}^{(1)} \cdot x_2 + w_{13}^{(1)} \cdot x_3 + b_1^{(1)} = 2$$

$$a_1^{(1)} = f(z_1^{(1)}) = \text{relu}(2) = 2$$

$$z_2^{(1)} = w_{21}^{(1)} \cdot x_1 + w_{22}^{(1)} \cdot x_2 + w_{23}^{(1)} \cdot x_3 + b_2^{(1)} = -1$$

$$a_2^{(1)} = f(z_2^{(1)}) = \text{relu}(-1) = 0$$

$$z_1^{(2)} = w_{11}^{(2)} \cdot a_1^{(1)} + w_{12}^{(2)} \cdot a_2^{(1)} + b_1^{(2)} = 0$$

$$a_1^{(2)} = f(z_1^{(2)}) = \text{relu}(0) = 0$$

$$z_2^{(2)} = w_{21}^{(2)} \cdot a_1^{(1)} + w_{22}^{(2)} \cdot a_2^{(1)} + b_2^{(2)} = 1$$

$$a_2^{(2)} = f(z_2^{(2)}) = \text{relu}(1) = 1$$

$$\hat{y} = w_{11}^{(3)} \cdot a_1^{(2)} + w_{12}^{(3)} \cdot a_2^{(2)} + b_1^{(3)} = 0$$

03. 题目如图，尝试求得 w_{21_grad} 。

```
import numpy as np
#定义样本数值
x=np.random.randn(10,2) #生成一个 10 行、2 列 的随机数二维数组
Y=np.random.randn(10) #生成一个 10 行、1 列 的随机一维数组
#参数初始化
w1=np.array([[1,1],[1,-1]])
w2=np.array([1,-1])
b1=np.ones((2,1)) #2 行 1 列
```

```

b2=np.ones(1)
lr=0.01
#定义 relu 函数和 relu 的导函数
def relu(z):
    return np.maximum(0, z)
def diff_relu(z):
    return (z > 0).astype(float)
#前向传播
z=w1@x.T+b1
a=relu (z) #激活函数处理
a_diff=diff_relu(z) #相当于 relu'(z)，即在 z 处的导函数
y=w2@a+b2
#求损失
loss=pow(y-Y,2)/2
loss.sum()
#求梯度
w11_grad=(y-Y)*w2[0]*1@x[:,0]
w21_grad=(y-Y)*w2[1]*a_diff[1,:]*x[:,0]
#更新参数
#print (z)
#print("this is w11_grad:",w11_grad)
print("this is w21_grad:",w21_grad)
#print (a,a_diff)

```

运行后：

```

[[ 2.40024975  2.33906643  2.01174911  0.02000143  0.36862652 -0.43458186
  1.44299365  0.56174837 -2.0901292  1.77799079]
 [-0.60902938  0.73189804  0.46740217  1.02312956  0.87591678  2.09234866
  1.5041538  1.1052584  -0.50260601  0.34549169]]
this is w11_grad: -2.607427050237053
this is w21_grad: -3.7180209358777023
[[ 2.40024975  2.33906643  2.01174911  0.02000143  0.36862652  0.
  1.44299365  0.56174837  0.
  1.77799079]
 [0.
  0.73189804  0.46740217  1.02312956  0.87591678  2.09234866
  1.5041538  1.1052584  0.
  0.34549169]] [[1. 1. 1. 1. 1. 0. 1. 1. 0. 1.]
 [0. 1. 1. 1. 1. 1. 1. 1. 0. 1.]]
PS C:\TZ\Code> python.exe "C:/TZ/Code/神经网络_10 input.py"
this is w21_grad: 0.685008551247992
PS C:\TZ\Code>

```

04.题目如图：

求 $\frac{\partial J}{\partial w_{11}}, \frac{\partial J}{\partial b_1}$

更新 w_{11}, b_1

盲求 $\hat{y}$ 具体值

$$z_1^{(1)} = 3, a_1^{(1)} = 3$$

$$z_2^{(1)} = 1, a_2^{(1)} = 1$$

$$z_3^{(1)} = 3, a_3^{(1)} = 3$$

$$z_1^{(2)} = 4, a_1^{(2)} = 4$$

$$z_2^{(2)} = -4, a_2^{(2)} = 0$$

$$\hat{y} = 1 \times 4 + 0 \times (-4) + 1 = 5$$

$$(y - \hat{y}) = (5 - 0) = 5$$

$$= \frac{\partial J}{\partial w_{12}^{(1)}} \cdot \frac{\partial J}{\partial b_1^{(2)}}$$

$z_3^{(1)}$ 与 $\hat{y}$ 之间有 2 条路径

$$w_{12}^{(1)} \rightarrow z_1^{(1)} \dots a_1^{(1)} \dots z_1^{(2)} \dots a_1^{(2)} \dots \hat{y} \rightarrow J$$

$$w_{12}^{(1)} \rightarrow z_3^{(1)} \dots a_3^{(1)} \dots z_3^{(2)} \dots a_3^{(2)} \dots \hat{y} \rightarrow J$$

$$\begin{aligned} \frac{\partial J}{\partial w_{12}^{(1)}} &= \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \cdot \frac{\partial z_1^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \cdot \frac{\partial z_1^{(1)}}{\partial w_{12}^{(1)}} \\ &+ \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_3^{(2)}} \cdot \frac{\partial a_3^{(2)}}{\partial z_3^{(2)}} \cdot \frac{\partial z_3^{(2)}}{\partial a_3^{(1)}} \cdot \frac{\partial a_3^{(1)}}{\partial z_3^{(1)}} \cdot \frac{\partial z_3^{(1)}}{\partial w_{12}^{(1)}} \end{aligned}$$

$$= (y - \hat{y}) \cdot w_{11}^{(3)} \cdot f'(z_1^{(3)}) \cdot w_{11}^{(2)} \cdot f'(z_1^{(2)}) \cdot x_2$$

$$+ (y - \hat{y}) \cdot w_{12}^{(3)} \cdot f'(z_2^{(3)}) \cdot w_{23}^{(2)} \cdot f'(z_3^{(2)}) \cdot x_2$$

$$= 5 \times 1 \times 1 \times 0 \times 1 \times 1 + 5 \times 0 \times 0 \times 0 \times 1 \times 0$$

$$= 0$$

$$\begin{aligned}
 b_1^{(2)} &\rightarrow z_1^{(2)} \dots a_1^{(2)} \dots \hat{y} \dots J \\
 \frac{\partial J}{\partial b_1^{(2)}} &= \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial z_1^{(2)}} \cdot \frac{\partial z_1^{(2)}}{\partial b_1^{(2)}} \\
 &= (\hat{y} - y) \cdot w_{11}^{(1)} \cdot f'(z_1^{(1)}) \cdot 1 \\
 &= 5 \times 1 \times 1 \times 1 \\
 &= 5
 \end{aligned}$$

更新 $w_{32}^{(1)}, b_1^{(2)}$

$$\begin{aligned}
 w_{32}^{(1)}(\text{新}) &= w_{32}^{(1)} - \text{Lr} \cdot \frac{\partial J}{\partial w_{32}^{(1)}} \\
 &= 0 - 0.01 \times 0 \\
 &= 0
 \end{aligned}$$

$$\begin{aligned}
 b_1^{(2)}(\text{新}) &= b_1^{(2)} - \text{Lr} \cdot \frac{\partial J}{\partial b_1^{(2)}} \\
 &= 1 - 0.01 \times 5 \\
 &= 0.95
 \end{aligned}$$

05.请简述设计神经网络中的输入和输出层时，应该注意什么。

输入层和输出层一般都只设计一层（但是隐藏层可以有多层），另外输入层可以有多个神经元，但是输出层一般只有一个神经元。

06.使用 Python 或 NumPy 手动编写两个函数：

1.激活函数 ReLU;

2.实现神经元的计算(包含加权求和及使用激活函数 ReLU);

并且尝试给定一个输入向量[1,0.5,-1]和相应的权重向量[0.5,0.25,-0.75]（没有偏置），计算神经元的输出。

答：

(1)relu 函数及其求导函数的代码实现

```

#定义 relu 函数
def relu(z):
    return np.maximum(0, z)
#定义 relu 函数的求导
def diff_relu(z):
    z = np.asarray(z)

```

```

    return (z > 0).astype(float)
print(relu(2), diff_relu(2))
print(relu(-1), diff_relu(-1))

```

运行后:

```

PS C:\TZ\Code> & "C:/Program Files/Python311/python.exe" c:/TZ/Code/week2_06_code.py
2 1.0
0 0.0
PS C:\TZ\Code> 

```

(2)

```

import numpy as np
#权重向量与输入向量赋值
w1=np.array([0.5,0.25,-0.75])
x=np.array([1,0.5,-1])
#定义 relu 函数
def relu(z):
    return np.maximum(0, z)
z=w1*x
y=relu (z) #激活函数处理
print(y)

```

运行结果:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\TZ\Code> & "C:/Program Files/Python311/python.exe" c:/TZ/Code/week2_06_code.py
[0.5  0.125 0.75 ]
PS C:\TZ\Code> 

```

07.请解释什么是前向传播和反向传播?两者在神经网络的训练过程中分别有什么作用?

从输入层到输出层方向的预测计算过程称为前向传播。

反向传播是求损失,然后利用链式法则求梯度,梯度下降后更新参数这样的逆向过程,也就是训练模型的过程。

08.请简述自动微分的工作原理,以及在网络训练过程中的作用。

自动微分就是对于隐藏层的“共用”部分如 $z_1^{(1)}$,如果 $z_1^{(1)}$ 上层有很多个神经元,那么

$\frac{\partial J}{\partial z_1^{(1)}}$ 只用算一次就可以减少很多重复计算。自动微分就是这种在 $\frac{\partial J}{\partial z_1^{(1)}}$ 节点上求完就

“存”的中间变量途径来求微分的原理。在网络训练过程中的作用就是对于超多参数的大模型，具备多个隐藏层并且每个层下的神经元很多个的情况下，可以大大加快计算速度。

09.尝试编写代码完成小批量梯度下降，使用 MNIST 数据集训练模型进行手写数字识别(选做)。